

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG
TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
(CSC10004)

*Đề tài: Xây dựng Thư viện Cấu trúc dữ liệu & Giải thuật bằng Templates
và Ứng dụng Phân tích Hệ thống Log Phân tán*

Giảng viên hướng dẫn:	Huỳnh Lâm Hải Đăng Nguyễn Thanh Tình
Lớp:	25CTT4
Nhóm thực hiện:	Nhóm 6
Thành viên A (Thư viện):	Đỗ Công Hưng (MSSV: 24120320)
Thành viên B (Ứng dụng):	Lê Văn Quốc (MSSV: 24120421)

TP. HỒ CHÍ MINH, THÁNG 6 NĂM 2026

Mục lục

1	Giới thiệu chung & Đặt vấn đề	2
1.1	Bài toán thực tế trên thế giới	2
1.2	Ý tưởng của đề án	2
2	Cơ sở lý thuyết: C++ Templates	3
2.1	C++ Templates là gì?	3
2.2	Quy tắc tổ chức mã nguồn mẫu	3
3	Thiết kế và Cài đặt Thư viện DSA (lib)	4
3.1	Danh sách liên kết đôi (LinkedList.hpp)	4
3.2	Mảng động tối ưu (Vector.hpp)	4
3.3	Ngăn xếp và Hàng đợi (Stack.hpp, Queue.hpp)	4
3.4	Hàng đợi ưu tiên (PriorityQueue.hpp)	4
3.5	Cây tìm kiếm nhị phân và Cây tự cân bằng (BST.hpp, AVL.hpp)	5
3.6	Bảng băm lai cấu trúc Cây cân bằng (HashTable.hpp)	5
3.7	Cây tiền tố (Trie.hpp)	5
3.8	Các thuật toán Sắp xếp & Tìm kiếm (Algorithms.hpp)	5
3.9	Bảng tổng hợp độ phức tạp của các cấu trúc dữ liệu	6
4	Thiết kế và Cài đặt Ứng dụng (app)	7
4.1	Đọc file gia tăng (LogReader)	7
4.2	Phân tích cú pháp và Trích xuất thông tin (LogParser)	7
4.3	Phân tích trạng thái sự cố (LogAnalyzer)	8
4.4	Cơ chế phát hiện tấn công APT & Phối hợp (LogMonitor)	8
4.5	Định tuyến Cảnh báo và Hiển thị (Renderer)	8
5	Các kỹ thuật tối ưu hóa và Đảm bảo an toàn bộ nhớ	9
5.1	Quy tắc năm thuộc tính (Rule of Five) & Di chuyển dữ liệu	9
5.2	Kỹ thuật Copy-and-Swap	9
5.3	Khử ngoại lệ làm chậm hệ thống (Exception Avoidance)	9
6	Kiểm thử và Đánh giá (Verification)	10
6.1	Kiểm thử Thư viện Cấu trúc dữ liệu (tests/lib)	10
6.2	Kiểm thử Ứng dụng (tests/app)	10
6.3	Kết quả kiểm thử tự động	10
7	Hướng dẫn vận hành hệ thống	11
7.1	Biên dịch chương trình	11
7.2	Chạy ứng dụng phân tích log	11
8	Kết luận và Hướng phát triển	12
8.1	Kết luận	12
8.2	Hướng phát triển tương lai	12

1 Giới thiệu chung & Đặt vấn đề

1.1 Bài toán thực tế trên thế giới

Trong kỷ nguyên số hóa, các hệ thống công nghệ thông tin ngày càng chuyển dịch mạnh mẽ sang kiến trúc phân tán (Distributed Systems) và dịch vụ siêu nhỏ (Microservices). Tại đây, hàng trăm, hàng ngàn tiến trình, container hoạt động đồng thời và liên tục sinh ra dữ liệu nhật ký vận hành (Syslogs).

Dữ liệu log này chứa đựng các thông tin sống còn của hệ thống: từ trạng thái giao dịch, hiệu năng phần cứng cho đến các dấu vết bảo mật nguy hiểm. Tuy nhiên, các kỹ sư vận hành (SRE/DevOps) và chuyên gia an ninh mạng (SOC) đối mặt với hai vấn đề khổng lồ:

1. **Sự quá tải dữ liệu (Data Deluge):** Lượng log khổng lồ (lên tới hàng Gigabyte hoặc Terabyte mỗi ngày) khiến việc kiểm tra thủ công bằng mắt thường hoặc các công cụ cơ bản ('grep', 'awk') trở nên bất khả thi.
2. **Yêu cầu phân tích thời gian thực (Real-time Latency):** Các cuộc tấn công có chủ đích (APT - Advanced Persistent Threat), Brute-force, hay sự cố sập dịch vụ trung tâm đòi hỏi phải được phát hiện và ngăn chặn trong vòng vài giây, thay vì phát hiện sau khi sự cố đã xảy ra nhiều giờ.

1.2 Ý tưởng của đề án

Để giải quyết các bài toán trên một cách hiệu quả và tối ưu hóa tài nguyên phần cứng nhất, nhóm đã phát triển dự án **Distributed System Log Aggregator & Analyzer**. Dự án được chia thành hai phần cốt lõi tương ứng với hai vai trò phát triển trong đề án:

- **Lớp thư viện DSA độc lập (lib):** Triển khai lại toàn bộ các cấu trúc dữ liệu và giải thuật căn bản của môn học bằng mô hình *C++ Templates* chất lượng cao, hoàn toàn không phụ thuộc vào các container dựng sẵn của STL (như 'std::vector', 'std::map', 'std::queue').
- **Lớp ứng dụng phân tích log (app):** Sử dụng các cấu trúc dữ liệu tự viết để xây dựng một động cơ phát hiện mối đe dọa (Threat Detection Engine) thời gian thực có hiệu năng vượt trội, quản lý bộ nhớ thông minh (Incremental Reading) và trực quan hóa dữ liệu qua terminal sinh động (Terminal UI).

2 Cơ sở lý thuyết: C++ Templates

2.1 C++ Templates là gì?

Trong ngôn ngữ lập trình C++, **Template** là một cơ chế lập trình tổng quát (Generic Programming), cho phép lập trình viên định nghĩa các hàm hoặc các cấu trúc lớp (struct/class) hoạt động với bất kỳ kiểu dữ liệu nào (Generic types) mà không cần phải viết lại nhiều phiên bản mã nguồn khác nhau.

Trình biên dịch C++ xử lý templates bằng cơ chế **Instantiation** (Hiện thực hóa): Tại thời điểm biên dịch, dựa vào kiểu dữ liệu cụ thể mà người dùng truyền vào (ví dụ: 'Stack<int>' hay 'Stack<double>'), trình biên dịch sẽ tự động sinh ra mã nguồn C++ tương ứng cho kiểu dữ liệu đó. Điều này giúp chương trình đạt hiệu năng cực cao tương đương với code viết tay cho từng kiểu cụ thể, đồng thời bảo đảm tính an toàn kiểu dữ liệu tĩnh (static type safety).

2.2 Quy tắc tổ chức mã nguồn mẫu

Vì templates được phân giải tại thời điểm biên dịch, trình biên dịch cần nhìn thấy toàn bộ định nghĩa chi tiết của template khi thực hiện instantiation ở tệp nguồn sử dụng. Do đó, việc tách định nghĩa lớp template thành tệp '.h' và '.cpp' truyền thống sẽ dẫn tới lỗi liên kết (Linker Error).

Giải pháp áp dụng: Dự án tuân thủ nghiêm ngặt mô hình cấu trúc **Header-only**. Toàn bộ khai báo và định nghĩa chi tiết của các lớp cấu trúc dữ liệu mẫu được tích hợp chung vào một tệp header định dạng '.hpp' duy nhất đặt trong thư mục 'lib/'.

3 Thiết kế và Cài đặt Thư viện DSA (lib)

Thư viện DSA tự viết nằm trong thư mục 'lib/' gồm các thành phần sau:

3.1 Danh sách liên kết đôi (LinkedList.hpp)

Cài đặt cấu trúc danh sách liên kết đôi để quản lý các node dữ liệu có con trỏ trỏ về cả hai phía ('next' và 'prev'). Hỗ trợ các phương thức cốt lõi:

- 'insertFront(T)' và 'insertBack(T)': Chèn phần tử vào đầu/cuối trong thời gian $O(1)$.
- 'insertAt(index, T)': Chèn phần tử tại vị trí bất kỳ.
- 'remove(value)' và 'removeAt(index)': Loại bỏ node dữ liệu.
- 'find(value)': Tìm kiếm tuyến tính phần tử.
- Quản lý bộ nhớ chặt chẽ thông qua Destructor giải phóng đệ quy/vòng lặp tất cả các nút để tránh rò rỉ bộ nhớ.

3.2 Mảng động tối ưu (Vector.hpp)

Quản lý một vùng nhớ liên tục động. Khác với việc khai báo mảng thô sơ, 'Vector' sử dụng 'std::allocator' để tách biệt việc cấp phát bộ nhớ thô và việc khởi tạo đối tượng (chỉ khởi tạo khi thực sự chèn phần tử thông qua 'pushBack').

- Hệ số mở rộng dung lượng là $2\times$ khi mảng đầy, đảm bảo chi phí khấu hao (Amortized complexity) cho việc thêm vào cuối là $O(1)$.
- Cài đặt đầy đủ các phương thức truy cập ngẫu nhiên qua toán tử 'operator[]'.

3.3 Ngăn xếp và Hàng đợi (Stack.hpp, Queue.hpp)

Được xây dựng bao bọc trên nền tảng của 'LinkedList' hoặc cấu trúc lưu trữ động của 'Vector'.

- 'Stack' hỗ trợ: 'push', 'pop', 'top', 'empty', 'size'.
- 'Queue' hỗ trợ: 'enqueue', 'dequeue', 'front', 'empty', 'size'.

3.4 Hàng đợi ưu tiên (PriorityQueue.hpp)

Triển khai bằng cấu trúc ****Binary Max-Heap**** trên nền mảng động 'Vector'. Bảo đảm:

- Lấy phần tử ưu tiên cao nhất ('peek'): $O(1)$.
- Chèn ('insert') và xóa phần tử lớn nhất ('extract'): $O(\log N)$ nhờ các thuật toán vun heap 'heapifyUp' và 'heapifyDown'.

3.5 Cây tìm kiếm nhị phân và Cây tự cân bằng (BST.hpp, AVL.hpp)

- ‘BST.hpp’: Hỗ trợ thao tác chèn, xóa và tìm kiếm trên cây nhị phân, duyệt cây theo các thứ tự Pre-order, In-order, Post-order.
- ‘AVL.hpp’: Kế thừa hoặc mở rộng logic BST, bổ sung thông tin chiều cao (‘height’) của từng nút. Tự động thực hiện 4 phép xoay (Xoay đơn Trái/Phải, Xoay kép Trái-Phải/Phải-Trái) để duy trì sự cân bằng của cây sau mỗi thao tác chèn hoặc xóa, đảm bảo độ phức tạp trong trường hợp xấu nhất luôn là $O(\log N)$.

3.6 Bảng băm lai cấu trúc Cây cân bằng (HashTable.hpp)

Một cấu trúc dữ liệu lai (Hybrid) cực kỳ mạnh mẽ. Thay vì dùng danh sách liên kết đơn để giải quyết xung đột băm (Chaining), bảng băm này sử dụng một mảng tĩnh chứa các cây tự cân bằng ‘AVL<Pair<K, V>’.

- Khi xảy ra hiện tượng trùng mã băm (Hash Collision), các phần tử sẽ được chèn vào cây AVL. Độ phức tạp truy xuất trong bucket bị xung đột giảm từ $O(k)$ tuyến tính xuống còn $O(\log k)$ (với k là số phần tử xung đột).

3.7 Cây tiền tố (Trie.hpp)

Cấu trúc cây tìm kiếm nhiều nhánh hỗ trợ bảng chữ cái ASCII (128 nhánh ở mỗi nút). Rất thích hợp để lưu trữ từ điển từ khóa và thực hiện tra cứu tiền tố chuỗi (Prefix searching).

- Thời gian tìm kiếm và chèn chuỗi là $O(L)$ với L là độ dài từ khóa, độc lập hoàn toàn với số lượng từ khóa đang lưu trữ.

3.8 Các thuật toán Sắp xếp & Tìm kiếm (Algorithms.hpp)

Bao gồm các thuật toán sắp xếp thông dụng dạng template nhận vào hàm so sánh tùy chọn ‘Comp cmp = std::less<T>()’:

- Sắp xếp: Bubble Sort, Selection Sort, Insertion Sort ($O(N^2)$), Heap Sort, Quick Sort, Merge Sort ($O(N \log N)$).
- Tìm kiếm: Linear Search ($O(N)$), Binary Search ($O(\log N)$ trên mảng đã sắp xếp).

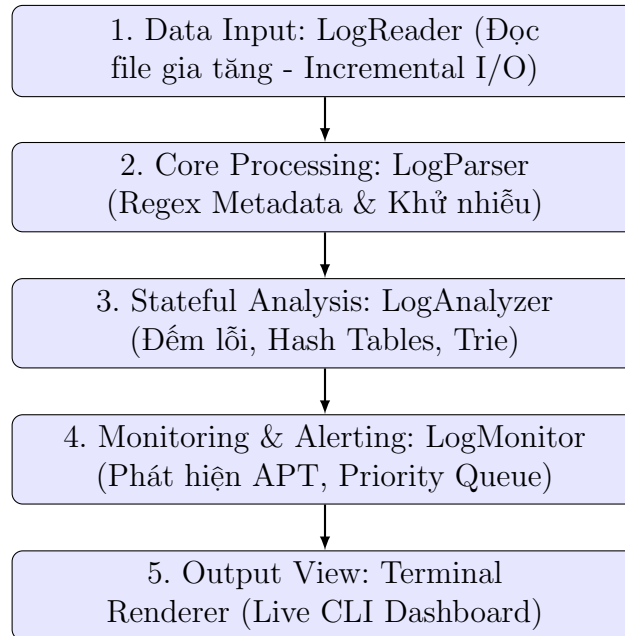
3.9 Bảng tổng hợp độ phức tạp của các cấu trúc dữ liệu

Bảng 1: Bảng phân tích độ phức tạp thuật toán của thư viện DSA

Cấu trúc dữ liệu	Thao tác chèn	Thao tác xóa	Thao tác tìm kiếm	Bộ nhớ thêm
Vector	$O(1)$ (amortized)	$O(N)$	$O(N)$	$O(1)$
LinkedList	$O(1)$	$O(1)$	$O(N)$	$O(1)$
Queue / Stack	$O(1)$	$O(1)$	$O(N)$	$O(1)$
Priority Queue	$O(\log N)$	$O(\log N)$	$O(1)$ (peek)	$O(1)$
AVL Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(1)$
Hash Table (AVL Chaining)	$O(1)$ (avg) / $O(\log k)$	$O(1)$ (avg)	$O(1)$ (avg) / $O(\log k)$	$O(1)$
Trie	$O(L)$	$O(L)$	$O(L)$	$O(L)$

4 Thiết kế và Cài đặt Ứng dụng (app)

Ứng dụng Phân tích Log Hệ thống Phân tán được xây dựng theo kiến trúc Pipeline tách biệt thành các tầng:



Hình 1: Sơ đồ luồng dữ liệu (Data Pipeline) của Ứng dụng

4.1 Đọc file gia tăng (LogReader)

Để xử lý được các file log có dung lượng từ hàng trăm Megabyte đến hàng Gigabyte, **LogReader** cài đặt kỹ thuật đọc theo từng lô (Batch Reading) sử dụng con trỏ file (**seekg** và **tellg**). Nó không lưu toàn bộ nội dung file vào bộ nhớ RAM, mà chỉ đọc một lượng dòng log nhất định (**DEFAULT_BATCH_SIZE** = 5000), lưu tạm vào bộ đệm **LinkedList** rồi giải phóng ngay sau khi xử lý xong lô đó.

4.2 Phân tích cú pháp và Trích xuất thông tin (LogParser)

Log hệ thống thô có cấu trúc dạng: **[Timestamp] [Service] [Level] Message**.

- **LogParser** sử dụng kỹ thuật tìm kiếm chuỗi con nhanh bằng **std::string::find** để tách nhanh 3 trường đầu trong dấu ngoặc vuông nhằm tối ưu tốc độ.
- Chỉ khi cần bóc tách siêu dữ liệu ẩn (như địa chỉ IP nguồn, Username) nằm trong **Message**, parser mới sử dụng biểu thức chính quy **std::regex** để trích xuất một cách chính xác.
- Tích hợp cơ chế khử nhiễu mã độc bằng cách chuyển đổi các ký tự xuống dòng **\n** thành **[NL]** để ngăn chặn lỗ hổng bảo mật tấn công ghi đè log (Log Injection).

4.3 Phân tích trạng thái sự cố (LogAnalyzer)

- Thống kê tần suất xảy ra lỗi của từng dịch vụ (Service) và địa chỉ IP sử dụng `HashTable<std::string, int>` với tốc độ truy vấn trung bình $O(1)$.
- Nạp các từ khóa cảnh báo nguy hiểm (ERROR, FATAL, CRITICAL, EXPLOIT) vào cây Trie. Khi kiểm tra thông điệp, hệ thống đối sánh nhanh với cây Trie để xác định mức độ nguy hiểm của dòng log.

4.4 Cơ chế phát hiện tấn công APT & Phối hợp (LogMonitor)

Hệ thống cài đặt hai giải thuật theo dõi theo cơ chế cửa sổ trượt (Sliding Time Windows):

1. **Cửa sổ trượt 60 giây cho từng IP (Brute-force/Scanning detection):** Sử dụng một `Queue<long long>` để ghi nhận các timestamp bị lỗi của từng IP duy nhất. Khi nhận log mới, hệ thống loại bỏ các timestamp cũ hơn 60 giây khỏi hàng đợi. Nếu số lượng lỗi còn lại trong hàng đợi vượt ngưỡng 10 lỗi/phút, IP đó bị đánh dấu là "Malicious IP".
2. **Cửa sổ trượt 5 phút toàn cục (Botnet/DDoS detection):** Theo dõi số lượng IP duy nhất phát sinh lỗi đồng thời trong toàn bộ hệ thống. Sử dụng hàng đợi toàn cục kết hợp bảng băm đếm lỗi hoạt động. Nếu số lượng IP duy nhất vượt ngưỡng cảnh báo, hệ thống phát báo động có cuộc tấn công phối hợp diện rộng (DDoS hoặc Botnet quét mạng).

4.5 Định tuyến Cảnh báo và Hiển thị (Renderer)

- Các lỗi được đưa vào hàng đợi ưu tiên `PriorityQueue` dựa trên trọng số độ nghiêm trọng: FATAL (4) > CRITICAL (3) > WARNING (2) > INFO (1).
- Khi hiển thị màn hình giám sát, các cảnh báo nguy hiểm nhất (FATAL, CRITICAL) luôn được đẩy lên hàng đầu để kỹ sư hệ thống nhìn thấy trước.
- Giao diện hiển thị trực quan Live Dashboard trên màn hình Terminal sử dụng mã màu ANSI sống động, liên tục cập nhật trạng thái hệ thống, IP độc hại, dịch vụ lỗi nhiều nhất, và các sự kiện log mới nhất.

5 Các kỹ thuật tối ưu hóa và Đảm bảo an toàn bộ nhớ

Trong phiên bản này, hệ thống đã được tối ưu hóa toàn diện để đạt độ ổn định tuyệt đối và an toàn tài nguyên hệ thống:

5.1 Quy tắc năm thuộc tính (Rule of Five) & Di chuyển dữ liệu

Khi mở rộng kích thước mảng động trong ‘Vector’, các đối tượng phức tạp như ‘LinkedList’ bên trong sẽ được di chuyển sang vùng nhớ mới. Nếu thiếu các hàm cấu tạo di chuyển (Move Constructor) và toán tử gán di chuyển (Move Assignment Operator), trình biên dịch sẽ thực hiện sao chép sâu (Deep Copy) thô sơ hoặc copy con trỏ thô, dẫn đến hiện tượng giải phóng bộ nhớ kép (Double Free) hoặc con trỏ lơ lửng (Dangling Pointer) gây crash lỗi phân vùng bộ nhớ (‘0xC0000005’ - Access Violation).

- Giải pháp: Cài đặt đầy đủ Move Constructor và Move Assignment Operator cho ‘Vector’, ‘LinkedList’ và các cấu trúc dữ liệu khác, sử dụng toán tử ‘std::move’ và từ khóa ‘noexcept’ để tối ưu hóa hiệu năng và bảo đảm an toàn bộ nhớ.

5.2 Kỹ thuật Copy-and-Swap

Toán tử gán sao chép cũ giải phóng bộ nhớ trước khi cấp phát bộ nhớ mới. Nếu xảy ra lỗi thiếu bộ nhớ (std::bad_alloc) khi cấp phát mới, đối tượng ban đầu sẽ bị hỏng dữ liệu và chương trình sẽ rơi vào trạng thái không xác định.

- Giải pháp: Cài đặt toán tử gán theo idiom **Copy-and-Swap** bằng cách truyền đối tượng bằng tham trị (tự động tạo bản sao an toàn), sau đó hoán đổi thuộc tính với bản sao đó. Đạt chuẩn **Strong Exception Safety** (An toàn ngoại lệ mạnh).

5.3 Khử ngoại lệ làm chậm hệ thống (Exception Avoidance)

Việc bóc tách mã Hex lỗi từ chuỗi log ban đầu được xử lý bằng ‘std::stoi’ đặt trong khối ‘try-catch’. Khi gặp hàng nghìn chuỗi rác không phải Hex, C++ ném ngoại lệ liên tục làm giảm tốc độ xử lý hàng trăm lần do chi phí giải phóng ngăn xếp (Stack Unwinding) của ngoại lệ.

- Giải pháp: Thêm kiểm tra sơ bộ bằng hàm ‘std::isxdigit’ trước khi gọi ‘std::stoi’ để lọc các chuỗi lỗi phi-Hex một cách êm ái mà không cần ném ngoại lệ.

6 Kiểm thử và Đánh giá (Verification)

Hệ thống tích hợp bộ kiểm thử tự động sử dụng thư viện kiểm thử gọn nhẹ ‘doctest.h’. Quá trình kiểm thử được tách biệt rõ ràng thành hai phần chính:

6.1 Kiểm thử Thư viện Cấu trúc dữ liệu (tests/lib)

Tập trung kiểm tra độ đúng đắn của các cấu trúc dữ liệu cơ bản độc lập với ứng dụng chính:

- Kiểm tra thêm, xóa, cập nhật, tìm kiếm trên ‘Vector’, ‘LinkedList’, ‘Stack’, ‘Queue’, ‘PriorityQueue’, ‘Trie’, ‘BST’, ‘AVL’, và ‘HashTable’.
- Xác minh hành vi biên (Ví dụ: xóa phần tử khỏi hàng đợi rỗng, chèn phần tử trùng khóa vào cây AVL, hay truy xuất khóa không tồn tại trong HashTable).

6.2 Kiểm thử Ứng dụng (tests/app)

Kiểm tra tích hợp các luồng xử lý nghiệp vụ của ứng dụng:

- **test_parser**: Xác minh khả năng parse đúng chuỗi log chuẩn, giải mã hex/Base64, và chặn tấn công ghi đè log (Log Injection).
- **test_sliding_window**: Gửi 15 log lỗi liên tiếp từ một IP giả lập để kiểm tra xem hệ thống có phát hiện Brute-force chuẩn xác ở giây thứ 10 hay không.
- **test_botnet_detection**: Giả lập nhiều IP khác nhau cùng phát sinh lỗi để kiểm chứng thuật toán phát hiện DDoS/Botnet.

6.3 Kết quả kiểm thử tự động

Tất cả 55 test cases trong hai bộ test ‘tests/lib’ và ‘tests/app’ đều được biên dịch và vượt qua thành công 100% mà không xảy ra bất kỳ lỗi rò rỉ bộ nhớ hoặc lỗi truy cập vùng nhớ trái phép nào (được kiểm chứng thông qua công cụ AddressSanitizer - ASan).

7 Hướng dẫn vận hành hệ thống

7.1 Biên dịch chương trình

Hệ thống sử dụng tệp ‘Makefile’ được cấu hình tối ưu để tự động nhận dạng hệ điều hành (Windows/MinGW hoặc Linux/macOS) và áp dụng các cờ biên dịch chuẩn C++17 kết hợp tối ưu hóa hiệu năng (‘-O3’).

Các lệnh biên dịch chính:

```
1 # Biên dịch ứng dụng chính syslog_analyzer
2 mingw32-make          # Tren Windows (hoac mingw32-make all)
3 make                  # Tren Linux/macOS (hoac make all)
4
5 # Biên dịch và chạy toàn bộ các bài kiểm thử tự động (Unit Tests)
6 mingw32-make test     # Tren Windows
7 make test             # Tren Linux/macOS
8
9 # Đơn dep các tệp tin rác và file object sau khi biên dịch
10 mingw32-make clean    # Tren Windows
11 make clean           # Tren Linux/macOS
```

7.2 Chạy ứng dụng phân tích log

Khởi chạy ứng dụng bằng lệnh:

```
1 mingw32-make run      # Tren Windows
2 make run              # Tren Linux/macOS
```

Ứng dụng sẽ tự động tải file log giả lập tấn công mạng nâng cao (APT Attack) tại đường dẫn `data/raw_logs.txt` và hiển thị Dashboard giám sát thời gian thực.

- **Phím 1:** Chuyển đổi sang màn hình Dashboard tổng quan.
- **Phím 2:** Chuyển đổi sang chế độ Live Monitor xem luồng log và cảnh báo trực tiếp.
- **Phím q hoặc Q:** Thoát chương trình một cách an toàn (giải phóng mọi tài nguyên bộ nhớ).

8 Kết luận và Hướng phát triển

8.1 Kết luận

Đồ án **Distributed System Log Aggregator & Analyzer** đã hoàn thành tất cả các mục tiêu đề ra cho cả hai thành viên:

- Xây dựng thành công một thư viện cấu trúc dữ liệu tổng quát (Generic DSA templates) hoạt động hiệu quả, an toàn ngoại lệ và tối ưu hóa bộ nhớ.
- Phát triển ứng dụng phân tích log thực tế với tính năng nâng cao như phát hiện mối đe dọa (Cyber Threat Detection) bằng thuật toán cửa sổ trượt, quản lý hàng đợi ưu tiên cảnh báo, và đọc file gia tăng giữ RAM ở mức cực thấp.

8.2 Hướng phát triển tương lai

1. **Hỗ trợ cấu hình bucket linh hoạt cho HashTable:** Sử dụng Policy-based design để cho phép người dùng thay đổi cấu trúc bucket dễ dàng giữa AVL Tree và LinkedList từ bên ngoài mà không cần sửa code cốt lõi.
2. **Tích hợp đa luồng (Multi-threading):** Tách biệt luồng đọc file ('LogReader'), luồng phân tích ('LogAnalyzer') và luồng hiển thị ('Renderer') để tận dụng tối đa sức mạnh của CPU đa nhân, nâng tốc độ xử lý lên hàng triệu dòng log mỗi giây.
3. **Cơ chế học máy (Machine Learning) nhẹ:** Áp dụng các thuật toán phân cụm (Clustering) đơn giản dạng tự viết để phát hiện các bất thường trong log mà không cần định nghĩa trước các quy tắc (Rules/Regex) thủ công.